影像處理實習

Week 7. 空間濾波 – 平滑濾波

助教 陳立堯

高斯平滑回顧

Part 1. 公式

二維高斯函數公式：

$$G(\mathrm{x},\mathrm{y}) = \frac{1}{2\pi\sigma^2}\mathrm{e}^{\left(-\frac{\mathrm{x}^2+\mathrm{y}^2}{2\sigma^2}\right)}$$

離散的卷積定義：

$$(I * K)(p) = \sum_q I(q)\,K(p-q)$$

高斯卷積公式：

$$G[I]_p = \frac{1}{W_p}\sum_{q\in S} G_{\sigma_s}(\|\mathbf{p}-\mathbf{q}\|)\,I_q$$

- 1/W : 將結果做歸一化
- p : 要計算的輸出像素位置
- q : 鄰近的像素座標
- ||p-q|| ：中心與鄰域的距離
- I(q) : 原始影像的亮度值

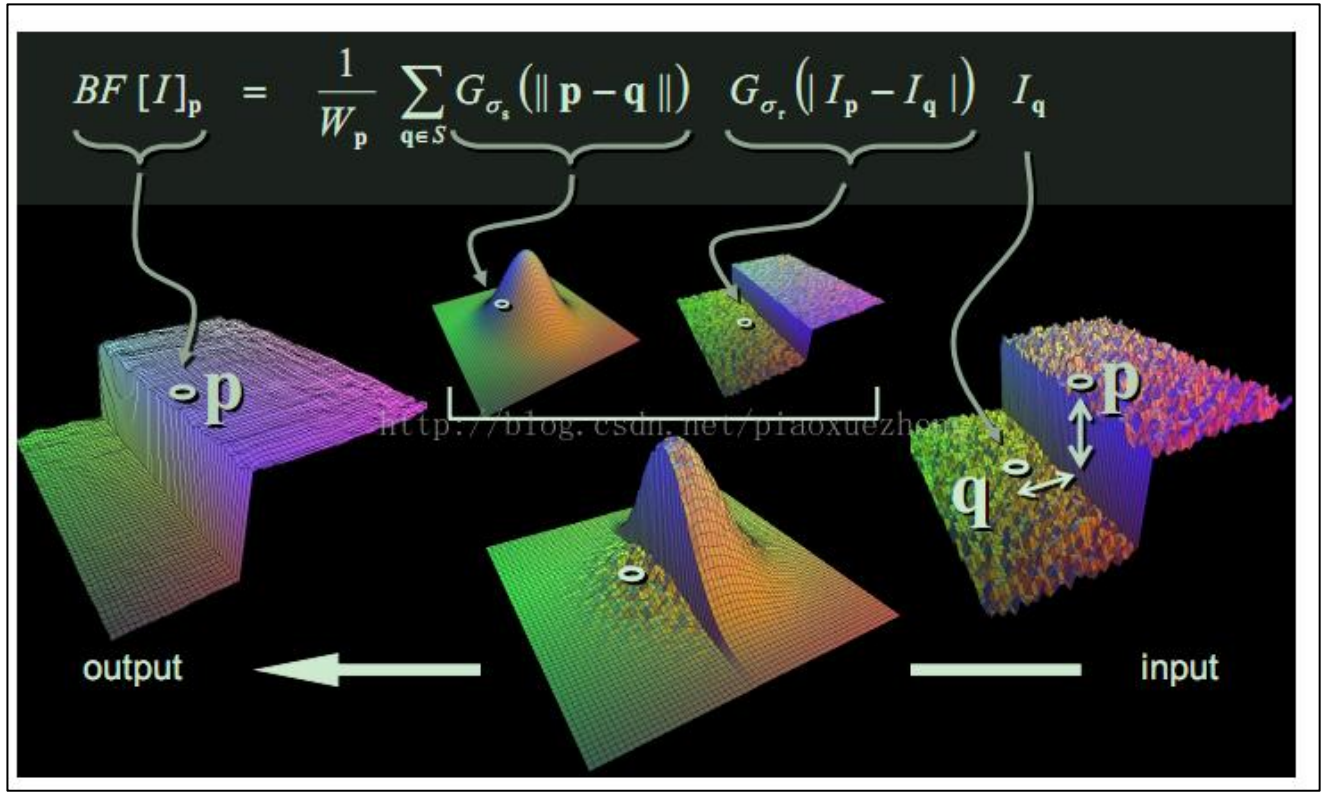
Part 1. 重點

1. σ標準差，控制「模糊程度」
2. 濾波器大小，一般取 $6\sigma + 1$
3. σ越大，模糊程度越強
4. 權重分佈連續，產生的平滑效果非常自然
5. 模糊邊界，不適合保留邊緣

雙邊濾波

Part 2. 雙邊濾波

出自IEEE 1998年的論文：Bilateral Filtering for Gray and Color Images



Part 2. 雙邊濾波

雙邊濾波是採用加權平均方法，是基於高斯分布的加權。雙邊濾波器考慮了歐式距離、像素間的輻射差異(相似程度、顏色強度、深度距離等)。在算中心像素值的時候，會考慮這兩個權重。

也可以說，他會同時考慮距離和顏色。

比起高斯濾波，他有時更能夠保留模糊後的邊緣

$$I_{\mathbf{p}}^{\mathrm{bf}} = \frac{1}{W_{\mathbf{p}}^{\mathrm{bf}}} \sum_{\mathbf{q}\in\mathcal{S}} G_{\sigma_s}(\|\mathbf{p}-\mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}}-I_{\mathbf{q}}|)\, I_{\mathbf{q}} \tag{1a}$$

$$\text{with} \quad W_{\mathbf{p}}^{\mathrm{bf}} = \sum_{\mathbf{q}\in\mathcal{S}} G_{\sigma_s}(\|\mathbf{p}-\mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}}-I_{\mathbf{q}}|) \tag{1b}$$

http://blog.csdn.net/piaoxuezhong

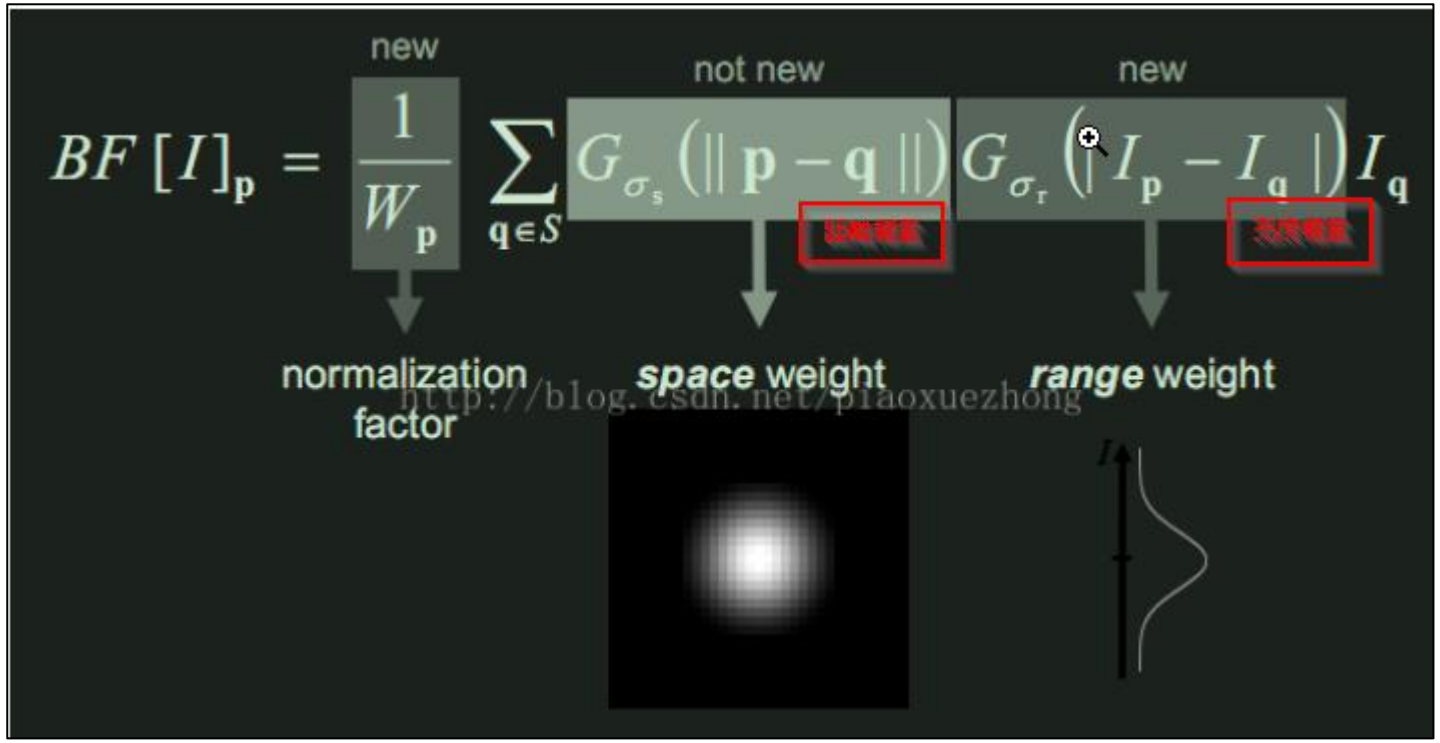
The parameter σ_s defines the size of the spatial neighborhood used to filter a pixel, and σ_r controls how much an adjacent pixel is downweighted because of the intensity difference. W^{bf} normalizes the sum of the weights.

Ipbf : 濾波後圖像

Iq : 輸入圖像

https://blog.csdn.net/piaoxuezhong/article/details/78302920

Part 2. 雙邊濾波


new
not new
new
$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in S} G_{\sigma_s}(\|\mathbf{p}-\mathbf{q}\|) G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|) I_{\mathbf{q}}$
normalization factor
space weight
range weight
http://blog.csdn.net/piaoxuezhong

Part 2. 雙邊濾波

1. 2.

$$w(i,j,k,l) = \exp\left(-\frac{(i-k)^2+(j-l)^2}{2\sigma_d^2} - \frac{\|I(i,j)-I(k,l)\|^2}{2\sigma_r^2}\right),$$

展開後可以簡寫合併成上方這個公式

1.

```
#建立空間權重
ax = np.arange(-bil_pad, bil_pad+1)
xx, yy = np.meshgrid(ax, ax)
spatial_weights = np.exp(-(xx**2 + yy**2) / (2 *
sigma_s ** 2))
```

2.

```
#計算亮度差異權重
intensity_diff = region - image[i, j]
range_weights = np.exp(-(intensity_diff**2) / (2 *
sigma_r**2))
```

Part 2. 雙邊濾波

$$w(i,j,k,l) = \exp\left(-\frac{(i-k)^2+(j-l)^2}{2\sigma_d^2} - \frac{\|I(i,j)-I(k,l)\|^2}{2\sigma_r^2}\right),$$

距離衰減　顏色值衰減

最後把兩個結果相乘，歸一化(/Wp)

```
weights = spatial_weights * range_weights
Wp = np.sum(weights)


bil_filter[i, j] = np.sum(region * weights) / Wp
```

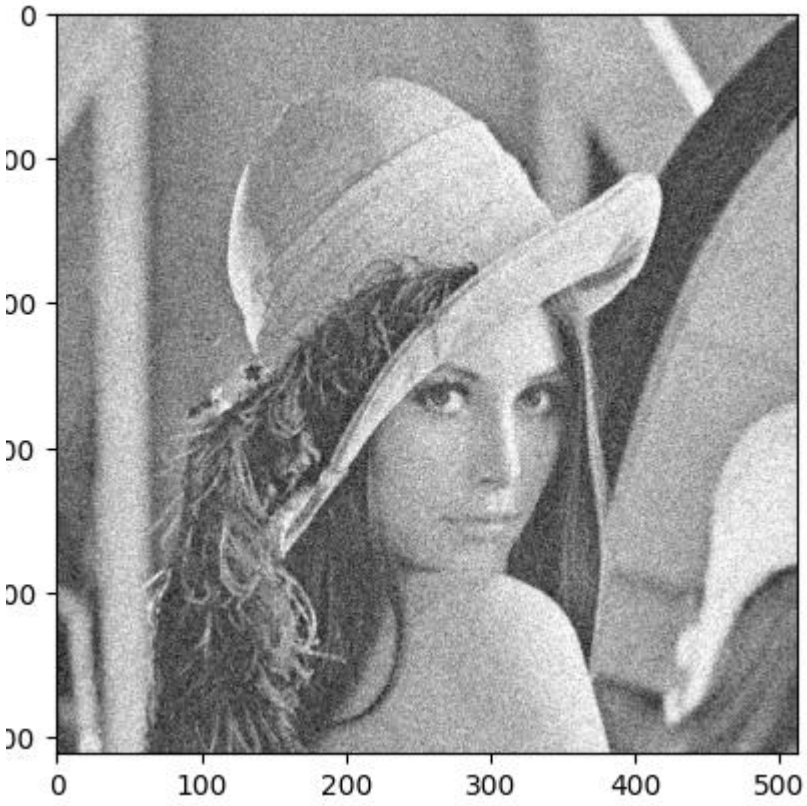
雜訊圖片製作

助教 陳立堯

Part 3.加入雜訊1 (高斯雜訊)

```
from skimage.util import random_noise

# 生成與影像相同大小的高斯雜訊
gaussian_noise = np.random.normal(20, 25, image.shape)

# 將雜訊加到影像上
noisy_image = image + gaussian_noise

# 限制像素值在 0-255 範圍內
noisy_image = np.clip(noisy_image, 0, 255)
plt.imshow(noisy_image, cmap='gray')
```

常見於感光元件雜訊、影像壓縮，整體均勻的顆粒感、毛玻璃感

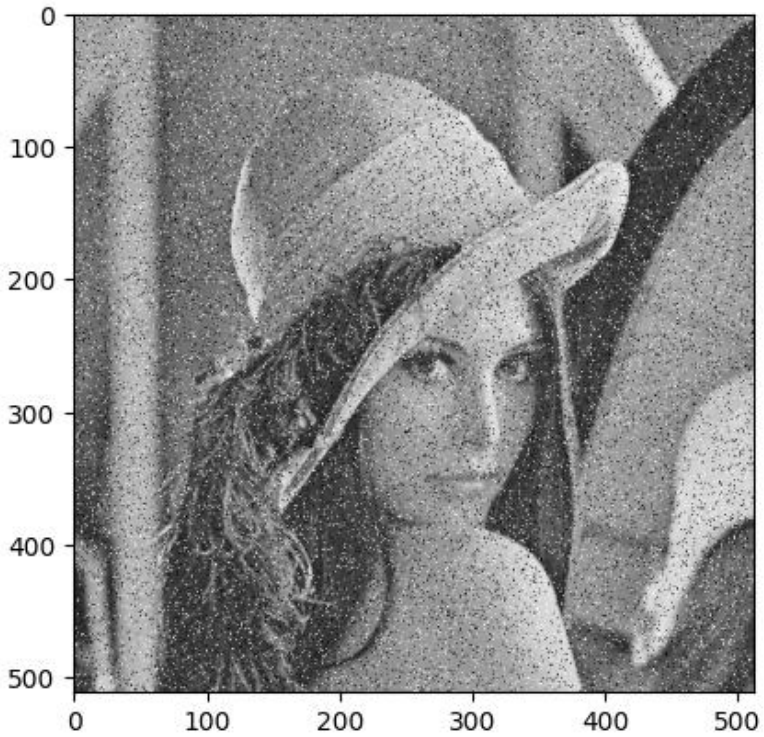


Part 3.加入雜訊2 (椒鹽雜訊)

```
noisy_image = image.copy()
salt_prob = 0.05
pepper_prob = 0.05

# 生成隨機矩陣
random_matrix = np.random.random(image.shape[:2])

# 添加鹽雜訊(白點 = 255)
noisy_image[random_matrix < salt_prob] = 255

# 添加椒雜訊(黑點 = 0)
noisy_image[random_matrix > 1 - pepper_prob] = 0
plt.imshow(noisy_image, cmap='gray')
```

發生於傳輸錯誤、硬體壞點。

是隨機的純黑、純白噪點



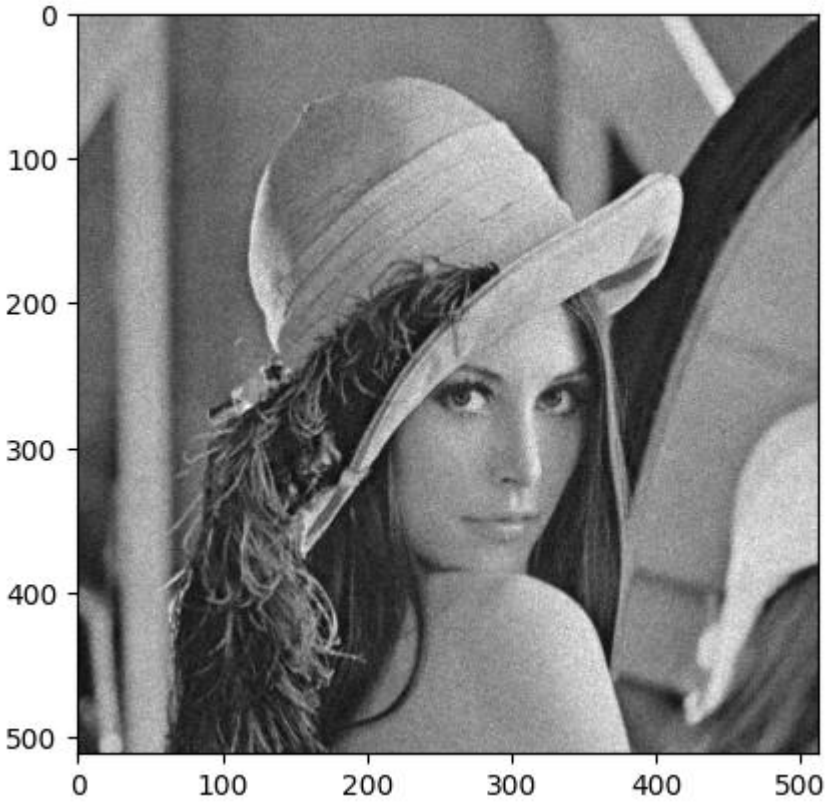
Part 3.加入雜訊3 泊松雜訊)

```
noisy = np.random.poisson(noisy_image)
noisy = np.clip(noisy, 0, 255)
```

泊松雜訊（**Poisson Noise**），又稱為 **Shot Noise（散粒雜訊）**，

主要出現在「光子或粒子」的隨機到達現象中。

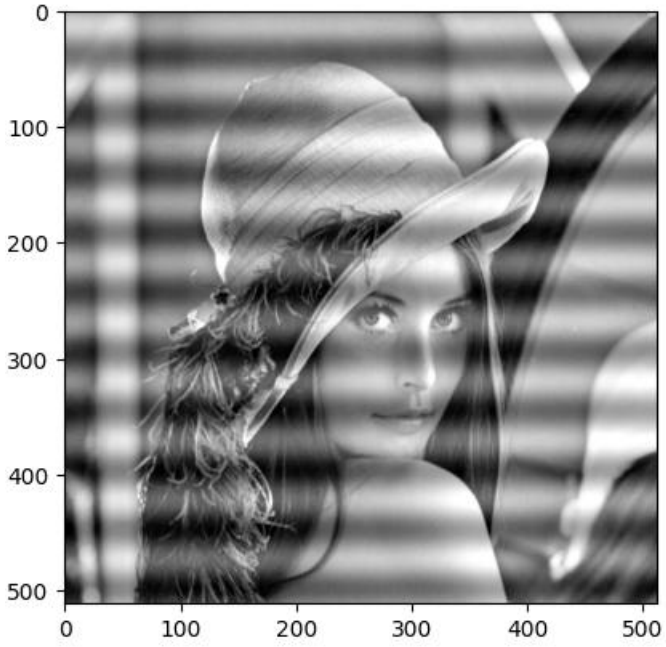
這種雜訊是**信號強度本身的隨機性**造成的，而非感測器誤差。



Part 3.加入雜訊4 週期性雜訊)

```
# 定義週期性雜訊參數
rows, cols = image.shape
A = 50              # 振幅
u0, v0 = 0, 10      # 垂直方向週期頻率（可調整觀察）

# 建立座標網格
x = np.arange(cols)
y = np.arange(rows)
X, Y = np.meshgrid(x, y)

# 生成週期性正弦雜訊
noise = A * np.sin(2 * np.pi * (u0 * X / cols + v0 * Y / rows))

# 將雜訊加入影像
noisy = image + noise
noisy = np.clip(noisy, 0, 255).astype(np.uint8)
```

影像中**重複出現的波狀干擾**，常由機械震動、電磁波干擾、掃描器抖動造成。



本日作業 (第七周)

1.找到3張圖生成雜訊 (可堆疊多種雜訊)
2.分別使用高斯濾波、雙邊濾波
3.比較兩者差異

將測試圖片、結果說明放到PDF or PPT